

Stack Overflow is a question and answer site for professional and enthusiast programmers. It's 100% free.

[Take the 2-minute tour](#)

Generating an n length bit list in Python

Work on work you love. From home.



 stackoverflowcareers

I want a function that will give me all the possible strings of a specified length that are composed of only zeros and ones. For example:

`spam(4)`

should get me:

```
['0110', '0111', '0001', '0011', '0010', '0101', '0100', '1110', '1100', '1101', '1010', '1011', '1001', '1000']
```

I tried to use `itertools.permutations` for the job. So, this is what I did.

```
def getPerms(n):
    perms = getCandidates(n)
    res = []
    for i in perms:
        res.extend(permutations(i))
    res = clean(res)
    return res

def clean(ar):
    res = []
    for i in ar:
        temp = ""
        for j in i:
            temp += j
        res.append(temp)
    return list(set(res))

def getCandidates(n):
    res = []
    for i in range(1, n):
        res.append("1"*i + "0"*(n-i))
    return res
```

But this is horribly inefficient and gives a Memory Error on 10 as input.

[python](#) [permutation](#) [bit](#)

edited Mar 23 '13 at 3:10



[nneonneo](#)

81.1k 14 91 171

asked Mar 23 '13 at 3:09



[Gerard](#)

204 2 10

3 To be clear -- you want it to contain at least one and at least one zero? Because `0000` and `1111` should be in your set otherwise. -- [nneonneo](#) Mar 23 '13 at 3:11

Yes, I need those possibilities. -- [Gerard](#) Mar 23 '13 at 14:27

3 Answers

You just want to generate bit-strings, evidently. Here's the fastest way I know:

```
for i in xrange(1, 2**n-1):
    yield '{:0{n}b}'.format(i, n=n)
```

This generates every bitstring of length exactly `n` containing at least one 1 and one 0.

Example:

```
>>> def gen(n):
...     for i in xrange(1, 2**n-1):
...         yield '{:0{n}b}'.format(i, n=n)
...
>>> list(gen(4))
['0001', '0010', '0011', '0100', '0101', '0110', '0111', '1000', '1001', '1010', '1011', '1100', '1101', '1110']
```

edited Mar 23 '13 at 3:21

answered Mar 23 '13 at 3:13



nneonneo

81.1k 14 91 171

Or just `['{:0{}b}'.format(i, n) for i in range(1 << n)]` – [georg](#) Mar 23 '13 at 9:41

Thank you! That works perfectly! – [Gerard](#) Mar 23 '13 at 14:23



Use `itertools.product` instead.

```
>>> import itertools
>>> [''.join(i) for i in itertools.product('01', repeat=4)]
['0000', '0001', '0010', '0011', '0100', '0101', '0110', '0111', '1000', '1001', '1010', '1011', '1100', '1101', '1110', '1111']
```

Using a function (assuming `itertools` has already been imported):

```
def bitGen(n):
    return [''.join(i) for i in itertools.product('01', repeat=n)]
```

For larger `n` s it might be more appropriate to return a generator.

```
def bitGen(n):
    return (''.join(i) for i in itertools.product('01', repeat=n))
```

Alternatively:

```
def bitGen(n):
    for i in itertools.product('01', repeat=n):
        yield ''.join(i)
```

edited Mar 23 '13 at 3:23

answered Mar 23 '13 at 3:14



Volatility

14k 3 28 59

Or `for i in itertools.product('01', repeat=n): yield ''.join(i)` in the last one. – [nneonneo](#) Mar 23 '13 at 3:20

@nneonneo thanks, I've put that in – [Volatility](#) Mar 23 '13 at 3:24

I didn't think about using `itertools.product()`. Thanks! – [Gerard](#) Mar 23 '13 at 14:24

In addition to the excellent answers above, if you want to continue down the path you started, here is a better implementation using `yield`:

```
from itertools import permutations

def spam(n):
    for perm in getPerms(n):
        print perm,
    print

def getPerms(n):
    for i in getCandidates(n):
        for perm in set(permutations(i)):
            yield ''.join(perm)

def getCandidates(n):
    for i in range(1, n):
        res = "1" * i + "0" * (n - i)
        yield res

spam(4)
```

answered Mar 23 '13 at 3:38



user650654

864 6 14